

Plant Sensor Project - Architecture Document

Comprehensive system architecture, component connectivity, sequence and flow diagrams

Generated: 2026-08-20 23:50:05

Repository: /Users/kanchan/Plant Sensor Project

Document location: plant-monitor/architecture/

Table of Contents

- [1. Executive Summary](#)
- [2. The Idea Behind the Project](#)
- [3. Component Inventory](#)
- [4. High-Level Architecture](#)
- [5. Hardware and Firmware Layer](#)
- [6. Serial Communication and Data Ingestion](#)
- [7. Continuous Monitoring and Alerts](#)
- [8. Touch-Triggered Workflow](#)
- [9. Pump Decision Logic](#)
- [10. Computer Vision and AI Layer](#)
- [11. Database Schema](#)
- [12. React Frontend Dashboard](#)
- [13. Logging and Storage](#)
- [14. Deployment and Startup](#)
- [15. Error Handling and Resilience](#)
- [16. API Reference](#)
- [17. Environment Variables](#)
- [18. Cross-Check Coverage Matrix](#)
- [19. Key File Inventory](#)

1. Executive Summary

The Plant Sensor Project is an end-to-end Internet of Things (IoT) plant monitoring and interactive care system. It combines an ESP32 microcontroller, environmental sensors, a capacitive touch sensor, a peristaltic pump, computer vision, and a local Vision-Language Model (VLM) to monitor a plant and react to human touch.

The core workflow is: the ESP32 continuously streams temperature and humidity data over USB serial. When a person touches the sensor, the system captures a webcam image, records a short video, runs YOLO person detection, optionally activates the pump when a person is detected, saves all media and metadata to a TimescaleDB time-series database, and queues a VLM analysis to describe the scene and plant health.

A FastAPI backend exposes REST endpoints and static media, while a React plus Vite dashboard visualizes live sensor data, historical charts, snapshots, video clips, AI analysis, and system logs.

All runtime data and logs can be stored on an externally mounted SD card to save laptop storage.

2. The Idea Behind the Project

The project explores three ideas in one device:

- **Always-on plant telemetry:** Track temperature and humidity around the plant over time and detect anomalies.
- **Human-plant interaction:** Use a touch sensor as a trigger. When a person touches the plant area, the system responds with audio feedback, captures what the camera sees, and decides whether to run the pump.
- **Local AI reasoning:** Instead of relying on the cloud, a local Ollama instance running the LLaVA vision model analyzes images and videos to detect people, describe clothing, assess plant health, identify yellowing, drooping, or wilting, and estimate image quality.

In the current demonstration configuration, the pump is activated when a person is detected in the image. This is controlled by the `TouchWorkflowOrchestrator` and the command `PUMP_ON_YELLOW_LEAVES`. The same VLM pipeline can later be switched to trigger on plant conditions such as yellow leaves, making the system a closed-loop plant-care assistant.

3. Component Inventory

Component	Type	Role	Key Path / Address
ESP32 Dev Board	Microcontroller	Reads sensors, drives pump and LEDs, streams serial data	/dev/cu.usbserial-0001, 115200 baud
Capacitive touch sensor	Sensor	Detects human touch on GPIO4	touch_sensor.ino / touch_sensor_led_pump_unified.ino
HDC302x	Sensor	Temperature and humidity over I2C (SDA GPIO21, SCL GPIO22)	plant-monitor/esp32/touch_sensor/touch_sensor.ino
Peristaltic pump	Actuator	Moves water when PUMP_ON_YELLOW_LEAVES is received	Relay GPIO on ESP32
Webcam	Camera	Provides images and video for analysis	OpenCV VideoCapture(0)
Host computer	macOS / Linux / Windows	Runs Python backend, database, VLM, frontend	localhost
TimescaleDB	Database	Time-series storage for sensors, snapshots, touch events, analysis	localhost:5433 / plantdb
FastAPI	Backend API	REST API, static media, status endpoints	plant-monitor/backend/app.py port 8000
Serial unified listener	Python service	Reads serial, parses data, triggers workflows, persists events	serial_unified_listener.py
Touch workflow orchestrator	Python service	Coordinates TTS, capture, YOLO, pump, VLM queue, DB save	services/touch_workflow.py
VLM worker	Python service	Background worker that claims queued snapshots and runs VLM	vlm_worker.py
Ollama	Local ML server	Serves llava:7b for image and video reasoning	localhost:11434
YOLOv8n	ML model	Lightweight person detection from Ultralytics	yolov8n.pt
React dashboard	Frontend	User interface for live data, snapshots, video, AI status	port 5173

Component	Type	Role	Key Path / Address
SD card storage	Storage	Optional external images, videos, and logs path	/Volumes/SD-128GB/PlantMonitor

4. High-Level Architecture

The system is organized in four layers: hardware/firmware, data ingestion and orchestration, AI and storage, and the web dashboard. The hardware layer streams data over USB serial. The ingestion layer parses, persists, and triggers actions. The AI layer adds person detection and VLM reasoning. The storage layer keeps time-series and media metadata. The dashboard queries the REST API to present data to the user.

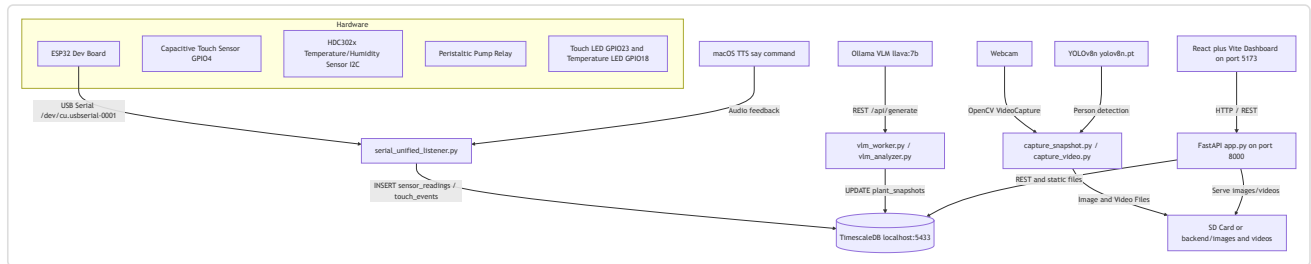


Figure 1: High-level system architecture

5. Hardware and Firmware Layer

The physical device is built around an ESP32 Dev Board. A capacitive touch sensor is wired to GPIO4 and debounced in firmware. The HDC302x temperature and humidity sensor connects to the default ESP32 I2C pins: SDA on GPIO21 and SCL on GPIO22. A peristaltic pump is driven through a relay pin, and two LEDs indicate touch and temperature states. The ESP32 communicates with the host over USB at 115200 baud.

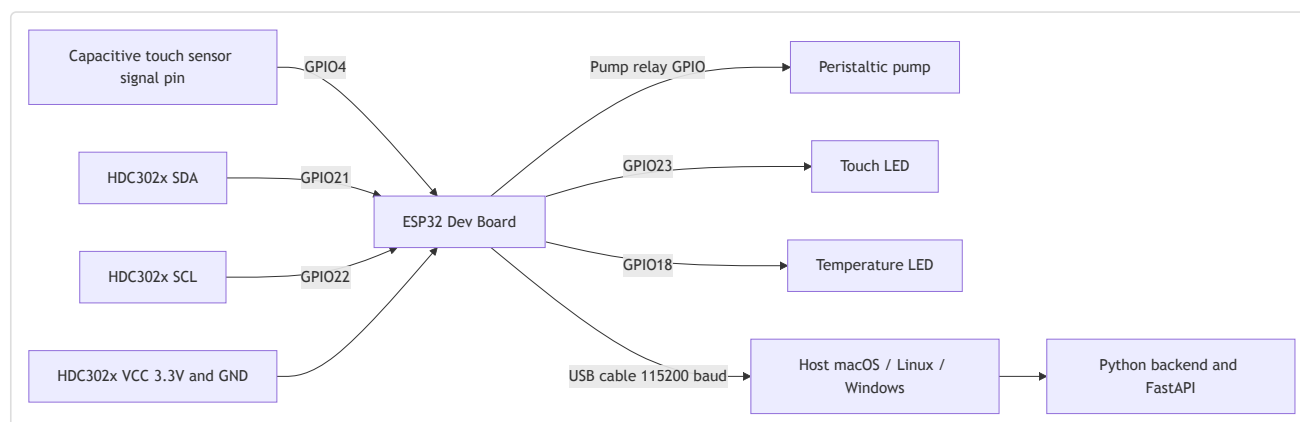


Figure 2: Hardware layer and wiring

The main production firmware is `plant-monitor/esp32/touch_sensor/touch_sensor.ino`. Additional prototype and test sketches live in the repository root, such as `touch_sensor/`, `touch_sensor_led_pump/`, `touch_sensor_led_pump_unified/`, `touch_led_control/`, and `touch_sensor_test/`. These were used to develop and isolate each subsystem before unification.

The unified firmware outputs two kinds of serial traffic:

- Plain-text lines `TOUCHED` and `NOT_TOUCHED` for touch events.
- JSON objects with fields `device_id`, `temperature_c`, `humidity_pct`, and `led_state` for periodic environmental readings.

The ESP32 also accepts the command `PUMP_ON_YELLOW_LEAVES` over the same serial link. On receipt, it turns the pump on for approximately 2 seconds and then enforces a 60-second cooldown before the next activation.

6. Serial Communication and Data Ingestion

The serial layer is the bridge between the ESP32 and the rest of the system. The primary process is `serial_unified_listener.py`, which:

- Scans for an ESP32 on `/dev/cu.usbserial-0001` and reconnects automatically if the port drops.
- Reads each serial line, distinguishes touch text from JSON sensor data, and keeps a reference to the serial port for downstream pump commands.
- Calls `insert_touch_event()` to write `TOUCHED` and `NOT_TOUCHED` events into the `touch_events` table with a timestamp and device ID.
- Calls `parse_sensor_data()` and `insert_sensor_reading()` to store temperature, humidity, and LED state in `sensor_readings`.
- Checks temperature thresholds and can trigger optional video recording.
- On touch, calls `handle_touch_event()`, which delegates to the `TouchWorkflowOrchestrator` running in a background thread.

Two older dedicated listeners, `serial_reader.py` and `serial_touch_listener.py`, still exist for testing and debugging but the unified listener is the preferred runtime path.

7. Continuous Monitoring and Alerts

Outside of touch events, the system continuously monitors the plant environment. The ESP32 sends a JSON reading at a regular interval. The unified listener inserts it and can trigger a temperature alert when the configured threshold is reached. The temperature threshold and snapshot cooldown are controlled through environment variables such as `SNAPSHOT_COOLDOWN` and `VIDEO_ALERT_*` values.

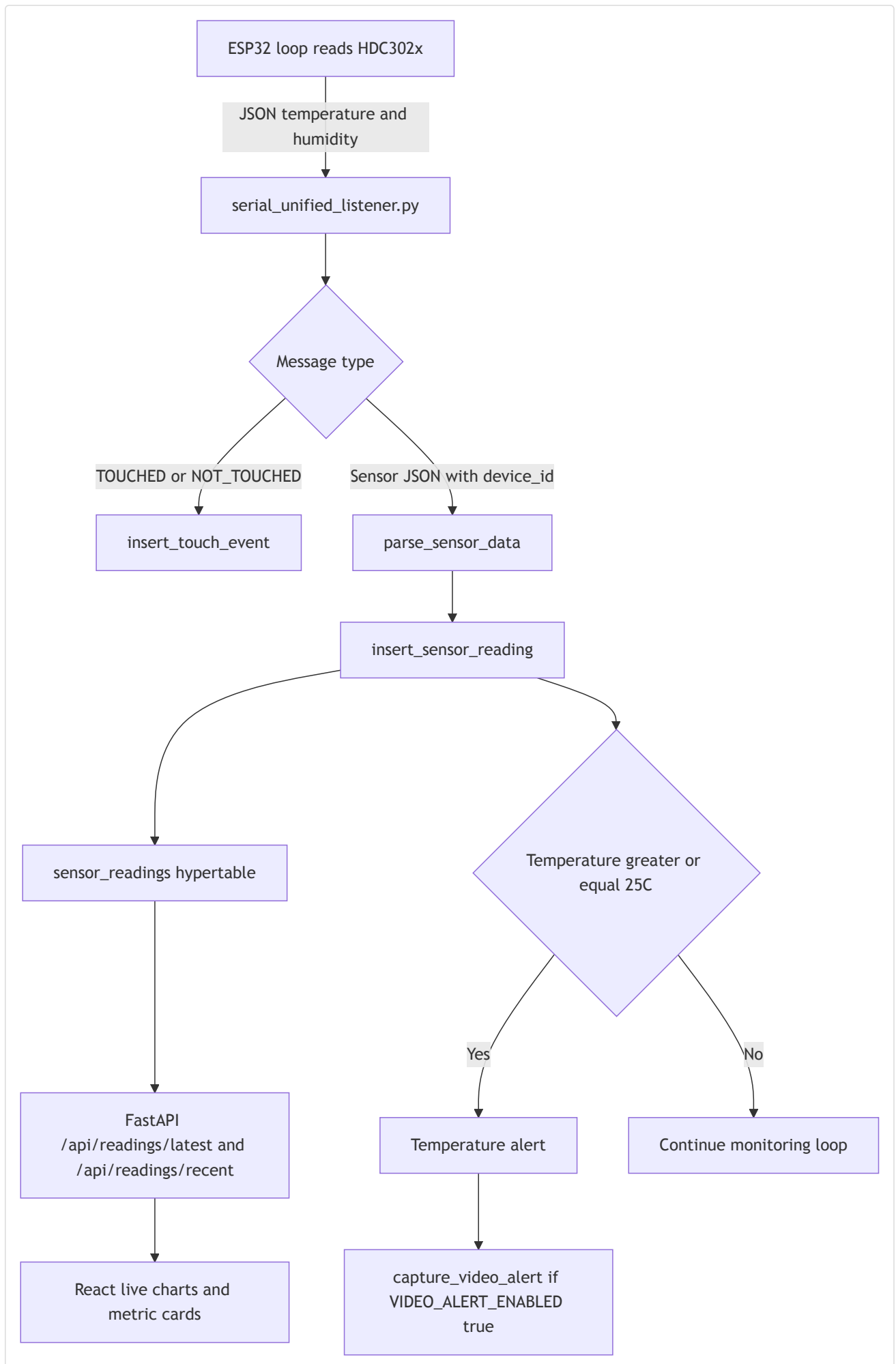


Figure 3: Continuous sensor monitoring and temperature alerts

React polls the `/api/readings/recent` and `/api/readings/latest` endpoints every few seconds, maps the data onto Recharts line charts, and displays the latest temperature, humidity, device ID, and last update time. The dashboard supports live mode with auto-refresh and custom time range queries.

8. Touch-Triggered Workflow

Touch is the main human interaction. When the firmware reports `TOUCHED`, the unified listener stores the event and starts the `TouchWorkflowOrchestrator`. The orchestrator runs the following steps in a background thread so the serial listener never blocks:

1. **Text-to-speech:** The `tts_service.py` calls the macOS `say` command to announce "Sensor touched".
2. **Snapshot:** `capture_webcam_image()` opens the webcam, applies exposure and brightness settings, captures a frame, and saves a JPEG file.
3. **Video clip:** `record_video_alert(duration=3, fps=10)` records a short MP4 clip by first writing an MJPG AVI and then converting it with `ffmpeg`.
4. **YOLO person detection:** `detect_person.py` loads the `yolov8n.pt` model, detects people, draws bounding boxes, and saves a boxed image and JSON metadata.
5. **Pump decision:** If a person is detected, the orchestrator sends `PUMP_ON_YELLOW_LEAVES` through the serial port with retry logic. Otherwise the pump stays off.
6. **Database insert:** `capture_with_vlm.py` or the orchestrator calls `insert_snapshot_quick()` to write the snapshot row including image path, video path, YOLO metadata, and an `analysis_status` of `queued`.

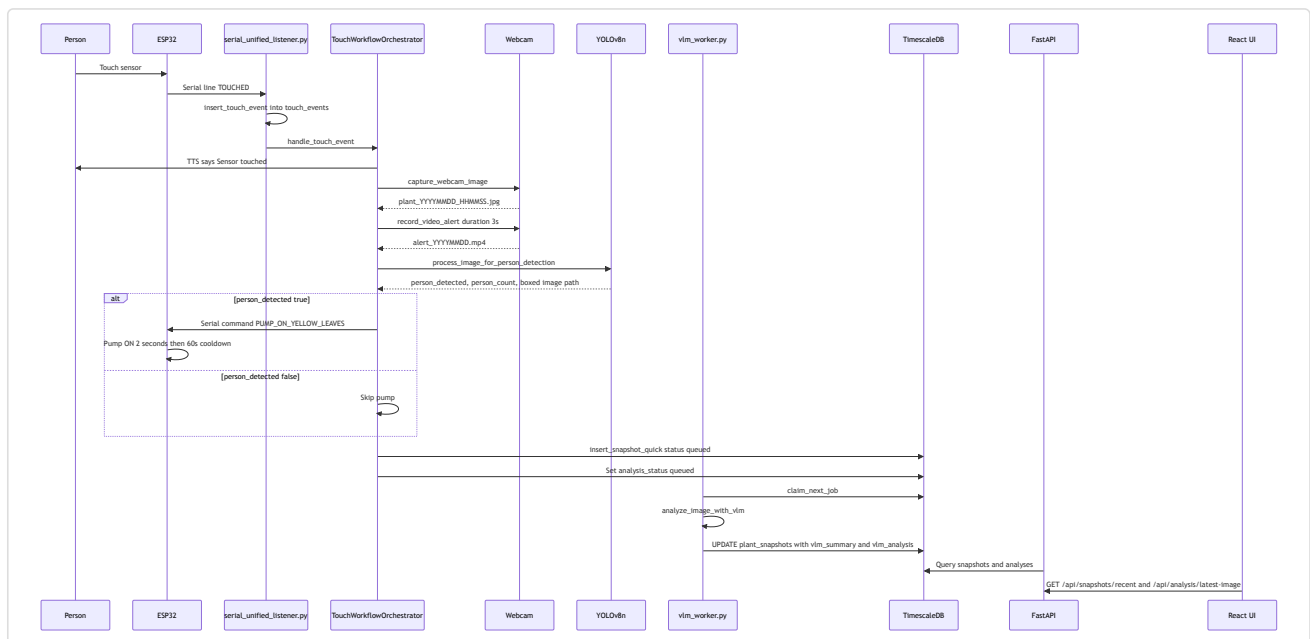


Figure 4: Touch-triggered end-to-end sequence

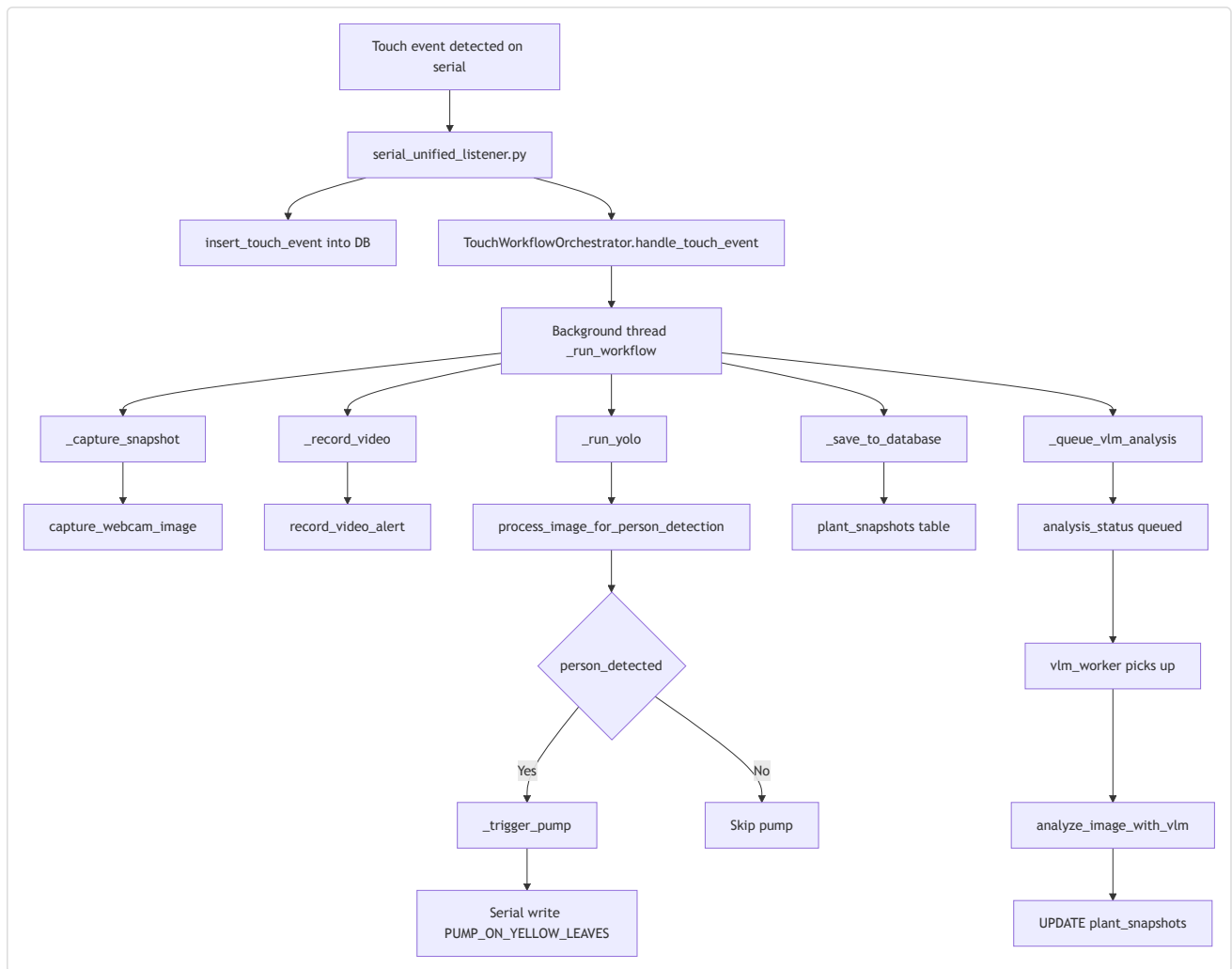
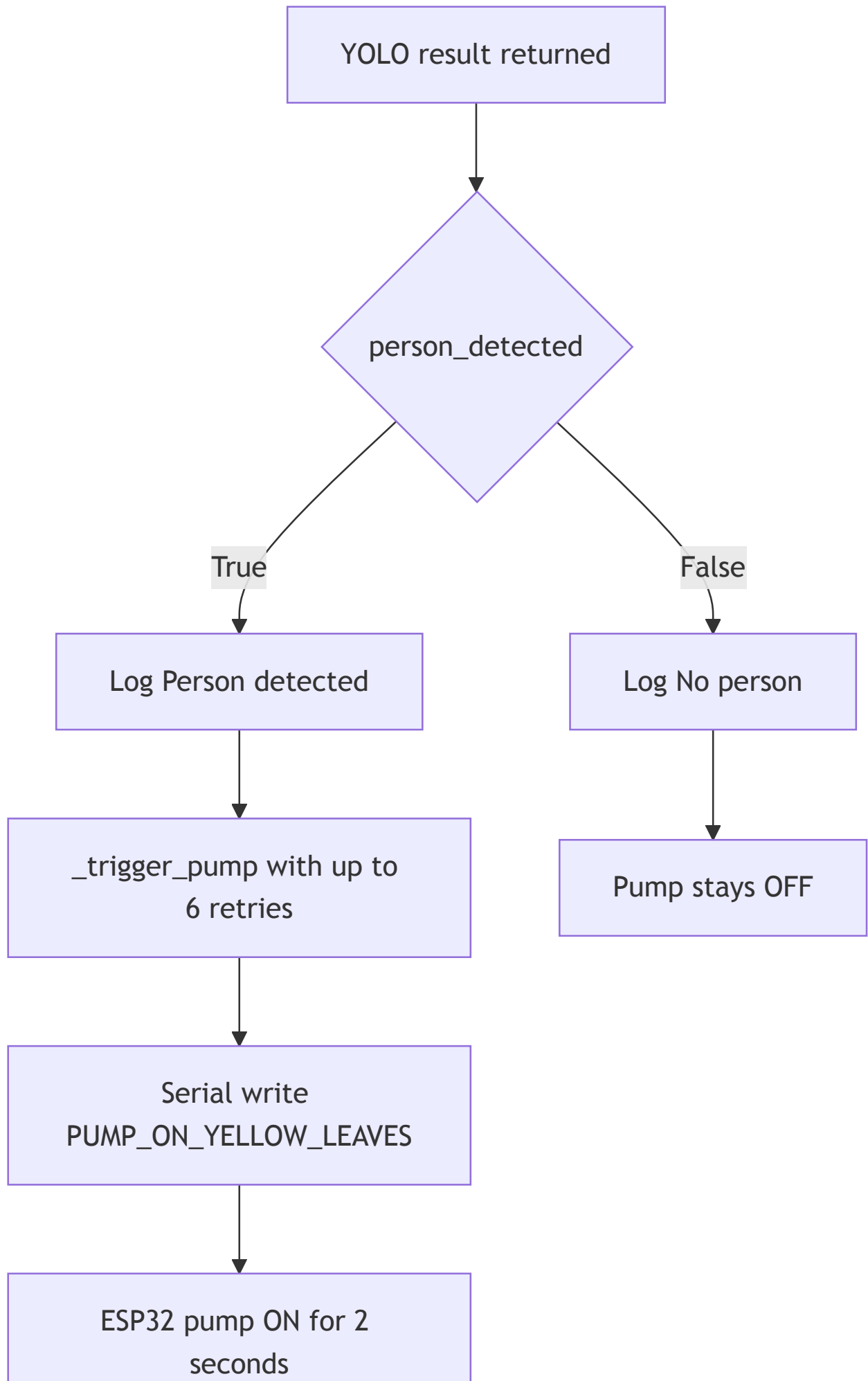


Figure 5: Touch workflow internal flow

9. Pump Decision Logic

The pump is controlled by the result of YOLO detection. The `_trigger_pump()` method in `services/touch_workflow.py` checks that the serial port is open, retries up to six times if the port is transiently unavailable, and then sends the `PUMP_ON_YELLOW_LEAVES` command followed by a flush. The ESP32 firmware handles the actual 2-second pump run and 60-second cooldown.



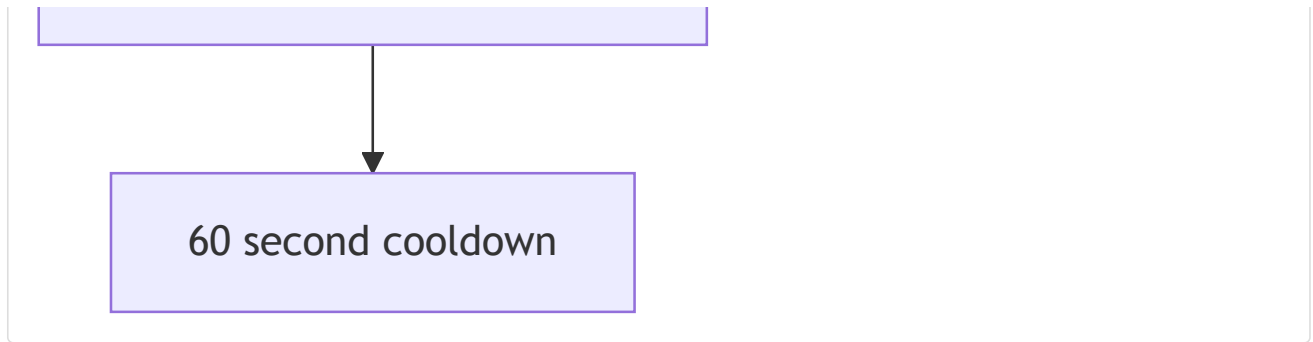


Figure 6: Pump decision logic

This behavior is currently configured for person-detection-based triggering as a demonstration. It can be extended to use VLM plant-health results by changing the condition that calls `_trigger_pump()`.

10. Computer Vision and AI Layer

The AI layer has two stages: fast local object detection and slower vision-language reasoning.

- **YOLOv8n:** The `detect_person.py` module uses the pretrained `yolov8n.pt` model from Ultralytics to detect people. It returns `person_detected`, `person_count`, a list of bounding boxes, and saves a boxed image for the UI.
- **Ollama VLM:** The `vlm.ollama_client.py` module connects to a local Ollama server at `OLLAMA_HOST` (default `http://localhost:11434`) using the `llava:7b` model. It base64-encodes an image, posts to `/api/generate`, and requests JSON output when `VLM_FORCE_JSON` is enabled.
- **VLM analyzer:** `vlm.vlm_analyzer.py` resizes the image for faster inference, selects a prompt, calls Ollama, and parses the returned JSON or extracts JSON from free text.
- **Analysis rules:** `vlm.analysis_rules.py` normalizes the VLM output and computes an enhanced dictionary with flags such as `plant_visible`, `yellowing_visible`, `drooping_visible`, `wilting_visible`, `image_quality`, and `reliability`.
- **VLM worker:** `vlm_worker.py` runs as a background daemon. It polls the `plant_snapshots` table for rows with `analysis_status = 'queued'`, claims the oldest one, runs `analyze_image_with_vlm()`, and updates the database with the results and `analysis_status = 'completed'` or `failed`.

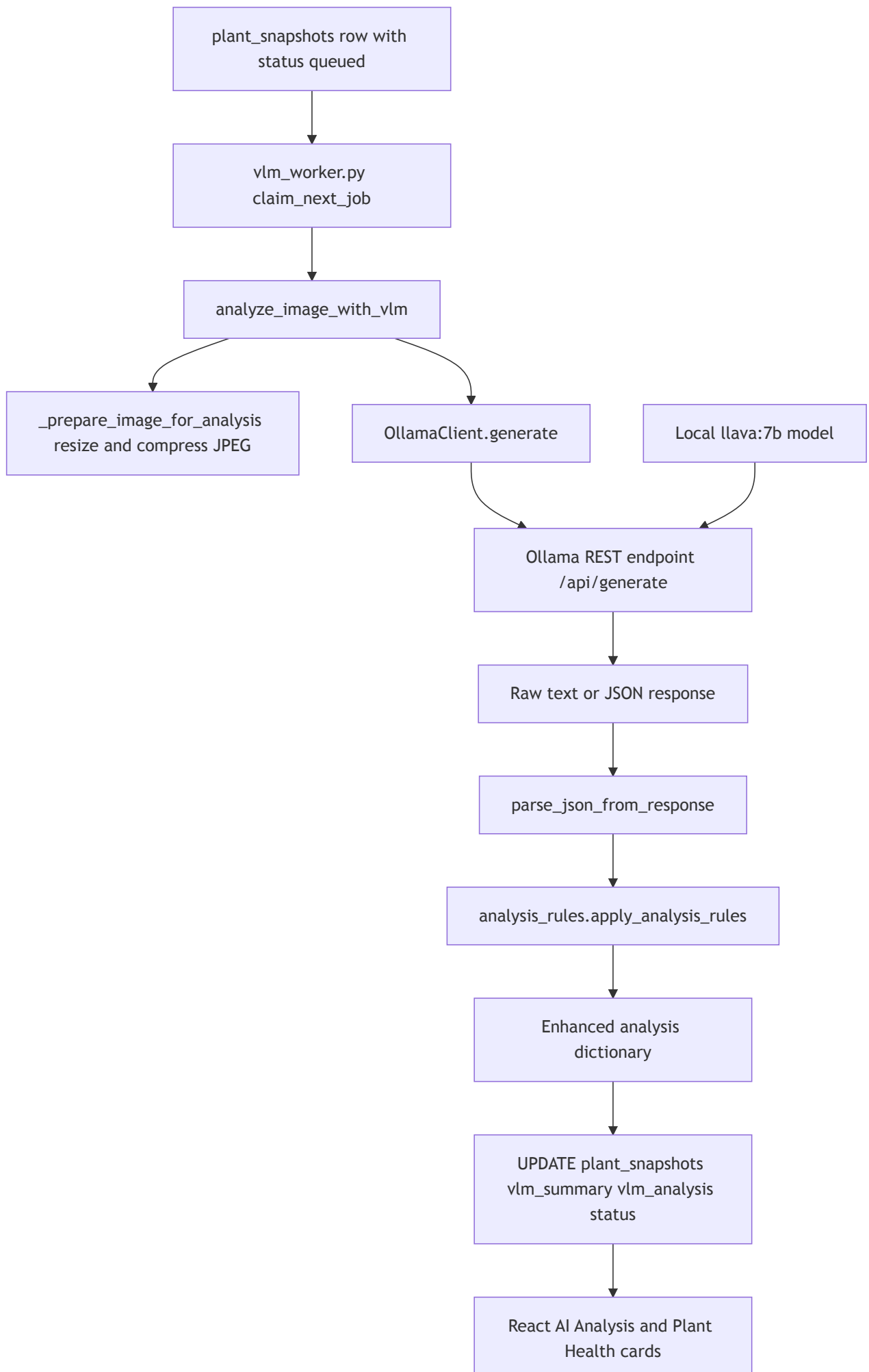


Figure 7: Vision-Language Model analysis pipeline

11. Database Schema

TimescaleDB is used for time-series storage. The backend uses `psycopg` and a connection string from `DATABASE_URL`. The core schema is built from `schema.sql` and incremental migrations in `backend/migrations/`.

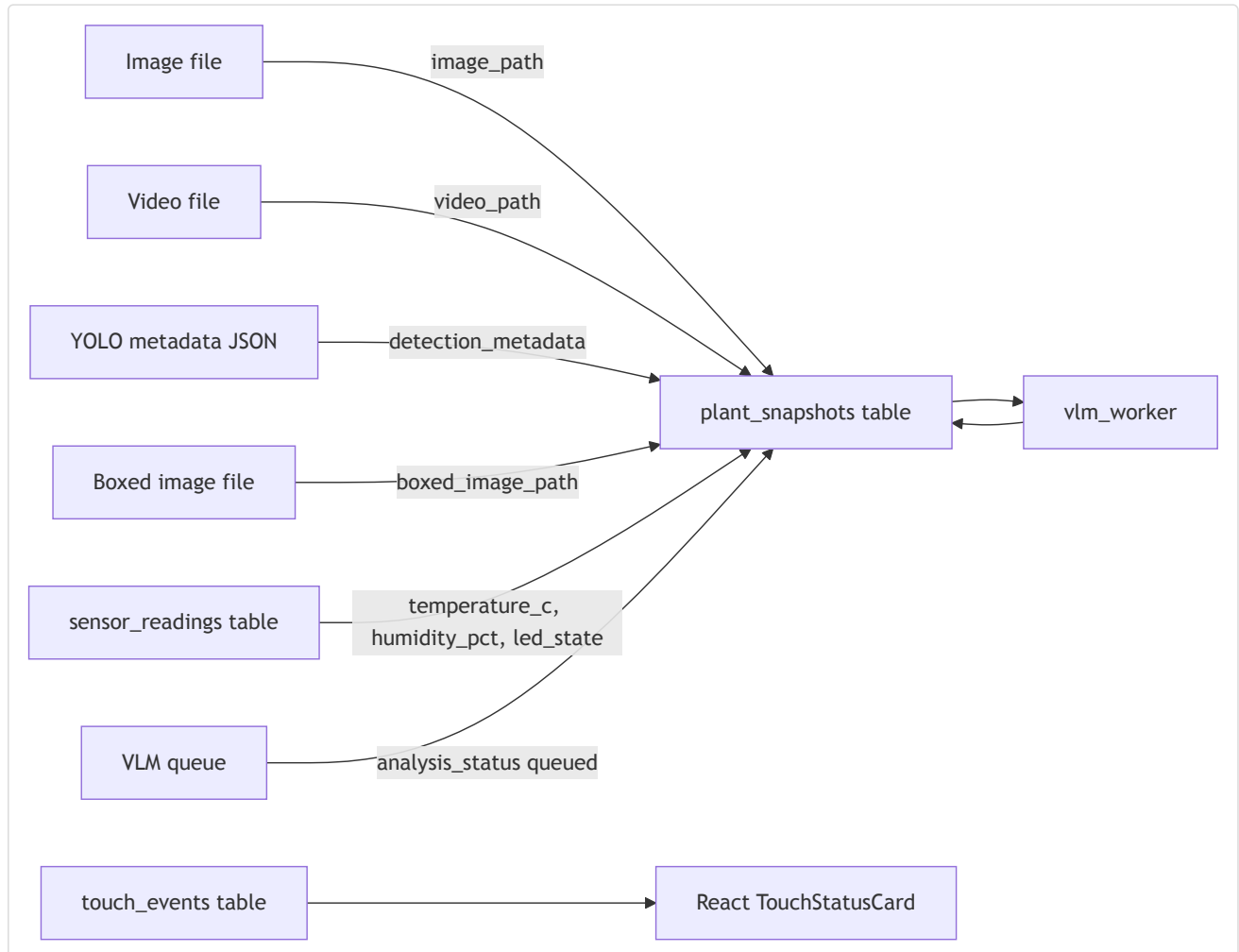


Figure 8: Database storage flow

Table	Purpose	Key columns
sensor_readings	Time-series temperature and humidity	time, device_id, temperature_c, humidity_pct, led_state
touch_events	Touch state transitions	id, timestamp, device_id, state
plant_snapshots	Captured images, YOLO and VLM metadata	time, image_path, boxed_image_path, video_path, temperature_c, humidity_pct, led_state, person_detected, person_count, detection_metadata, vlm_summary, vlm_analysis, plant_visible, plant_occluded, plant_health_guess, yellowing_visible, drooping_visible, wilting_visible, image_quality, analysis_status, analysis_error, analysis_reliability, vlm_model, analyzed_at

Table	Purpose	Key columns
video_analysis	VLM analysis of recorded video clips	id, time, video_path, snapshot_id, vlm_summary, vlm_analysis, person_entered, person_left, person_stayed, plant_touched, plant_blocked, plant_visible_throughout, significant_motion, motion_description, event_type, frames_analyzed, frame_paths, analysis_status, vlm_model, analyzed_at

Helpful views include `latest_analyzed_snapshots`, `plant_health_alerts`, and `video_events`. The React dashboard and FastAPI endpoints query these tables and views.

12. React Frontend Dashboard

The frontend is a React single-page application built with Vite. It fetches data from the FastAPI backend at `http://localhost:8000` and renders several focused cards.

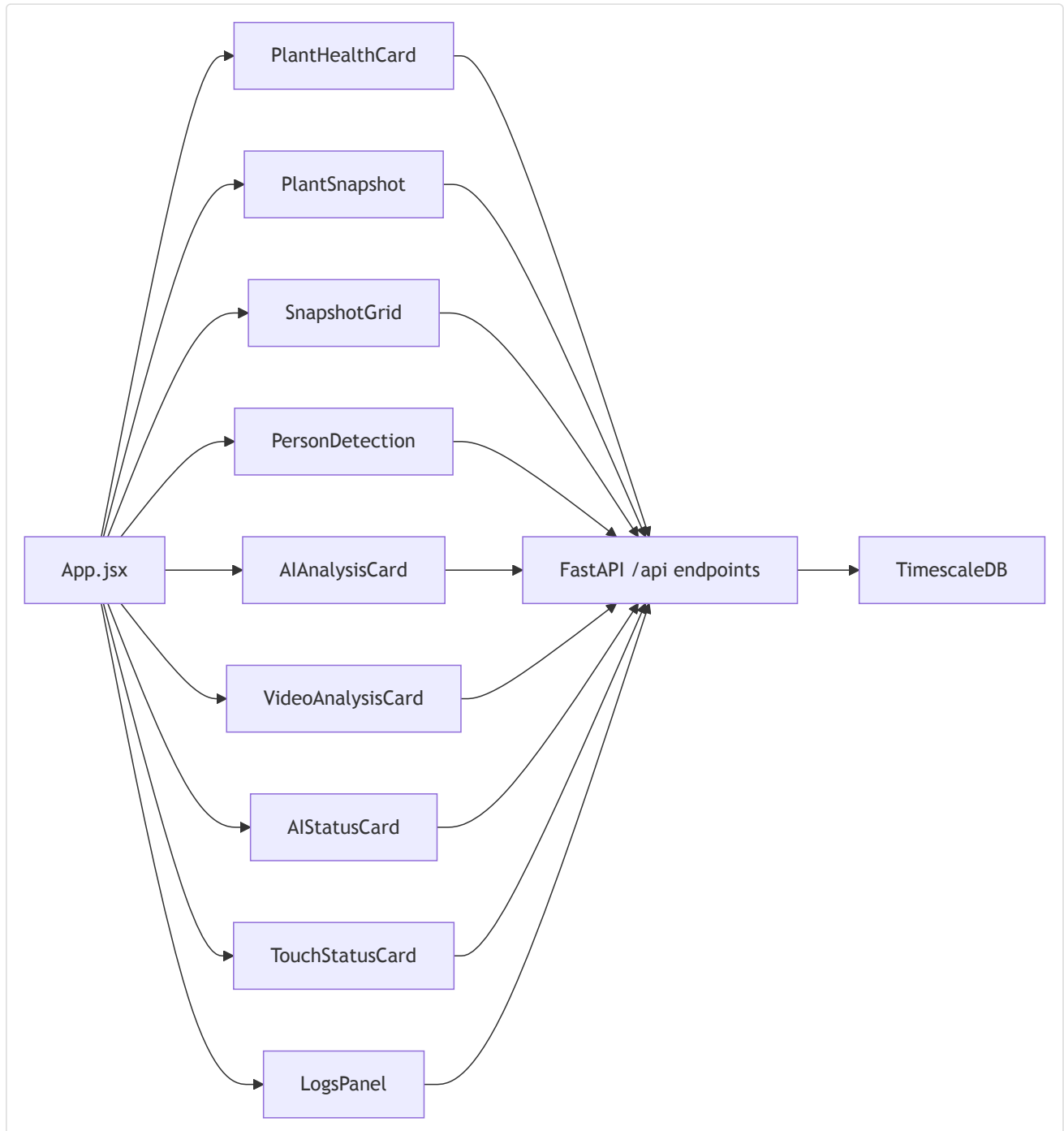


Figure 9: React frontend component flow

- **App.jsx:** Main layout, state for live/historical mode, date range picker, and fast-preset buttons (last 5 minutes, 15 minutes, 1 hour, today).
- **PlantHealthCard:** Displays the latest VLM plant health assessment and warning flags (yellowing, drooping, wilting).
- **PlantSnapshot:** Shows the most recent captured image with temperature and humidity overlay.

- **SnapshotGrid:** Gallery of recent snapshots with thumbnails, video badges, and click-to-detail behavior.
- **PersonDetection:** Highlights person detection metadata and boxed images.
- **AIAnalysisCard / VideoAnalysisCard:** Display VLM image and video analysis summaries.
- **AIStatusCard:** Shows Ollama connection, model name, queue status, and error counts.
- **TouchStatusCard:** Displays the latest touch state and seconds since last touch.
- **LogsPanel:** Auto-refreshing log viewer with level coloring, live indicator, and pause/resume controls.

Recharts is used for the temperature and humidity line chart. The dashboard uses CSS-in-JS styling and fetches images, boxed images, and videos through the FastAPI static and video endpoints.

13. Logging and Storage

Centralized logging is managed by `logging_config.py`. It creates rotating file handlers for each component and can mirror logs to the console. Log files are stored under the directory pointed to by `LOGS_DIR`. If an SD card is mounted at `/Volumes/SD-128GB/PlantMonitor` and writable, the start scripts set `LOGS_DIR`, `IMAGES_DIR`, and `VIDEOS_DIR` to that location. Otherwise they fall back to local `backend/logs`, `backend/images`, and `backend/videos`.

The Logs API in `routes/logs.py` lets the frontend and operators retrieve the latest lines, list log files, read an entire file, and get aggregate statistics. Typical loggers include `app`, `api`, `serial_reader`, `camera`, and `vlm`.

14. Deployment and Startup

All services can be started together with `start_all.sh` or individually through dedicated scripts.

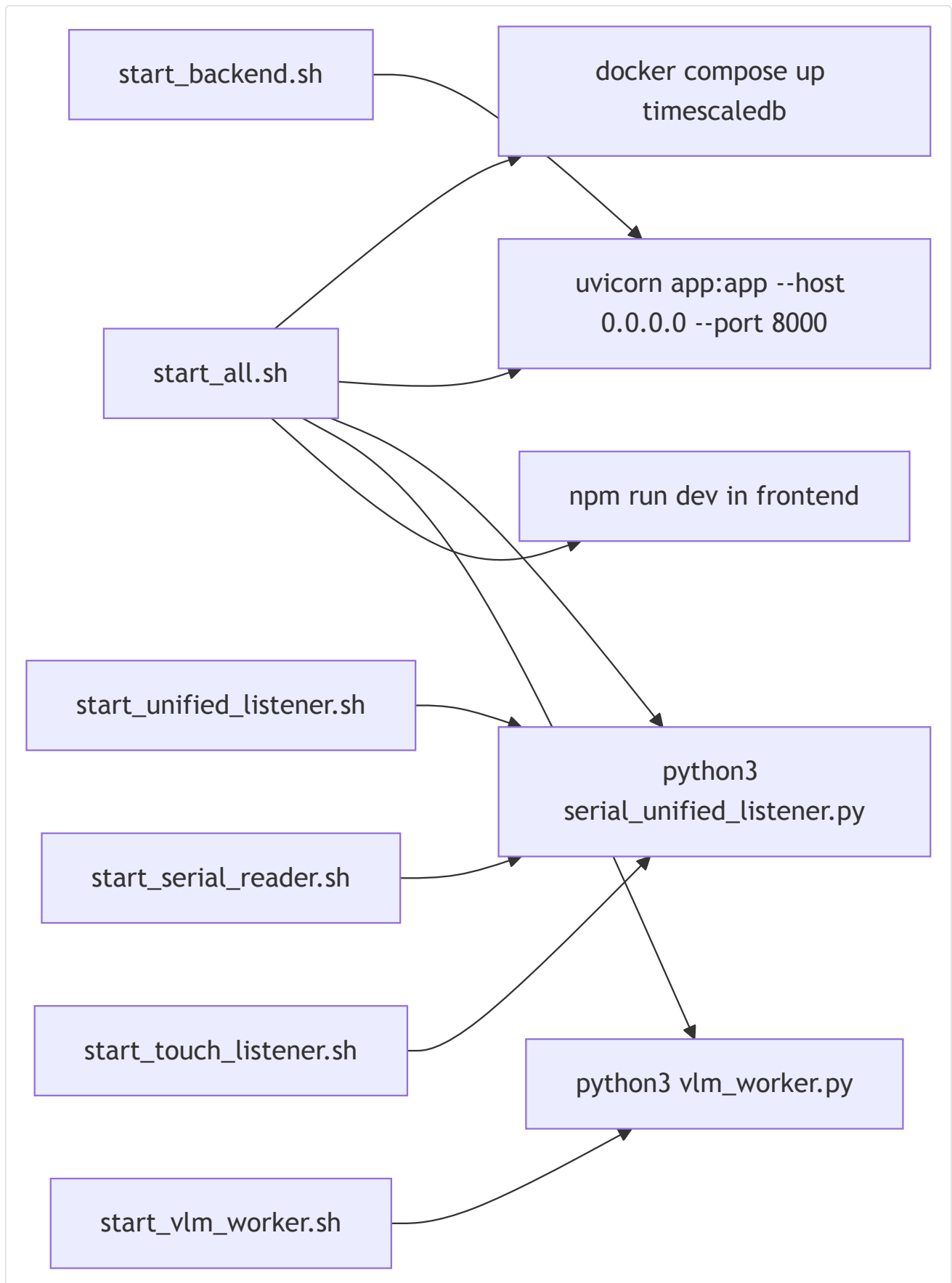


Figure 10: Deployment and startup scripts

Service	Script	Command / default port
TimescaleDB	docker-compose.yml or start_all.sh	localhost:5433

Service	Script	Command / default port
FastAPI	start_backend.sh, start_all.sh	0.0.0.0:8000
Unified serial listener	start_unified_listener.sh	Reads /dev/cu.usbserial-0001
VLM worker	start_vlm_worker.sh, start_all.sh	Polls DB and Ollama localhost:11434
React dev server	npm run dev	localhost:5173

All startup scripts try to activate a local Python virtual environment from `.venv` or `venv` and prefer the SD card paths when the card is mounted and writable.

15. Error Handling and Resilience

The system is designed to degrade gracefully when parts of the pipeline are unavailable.

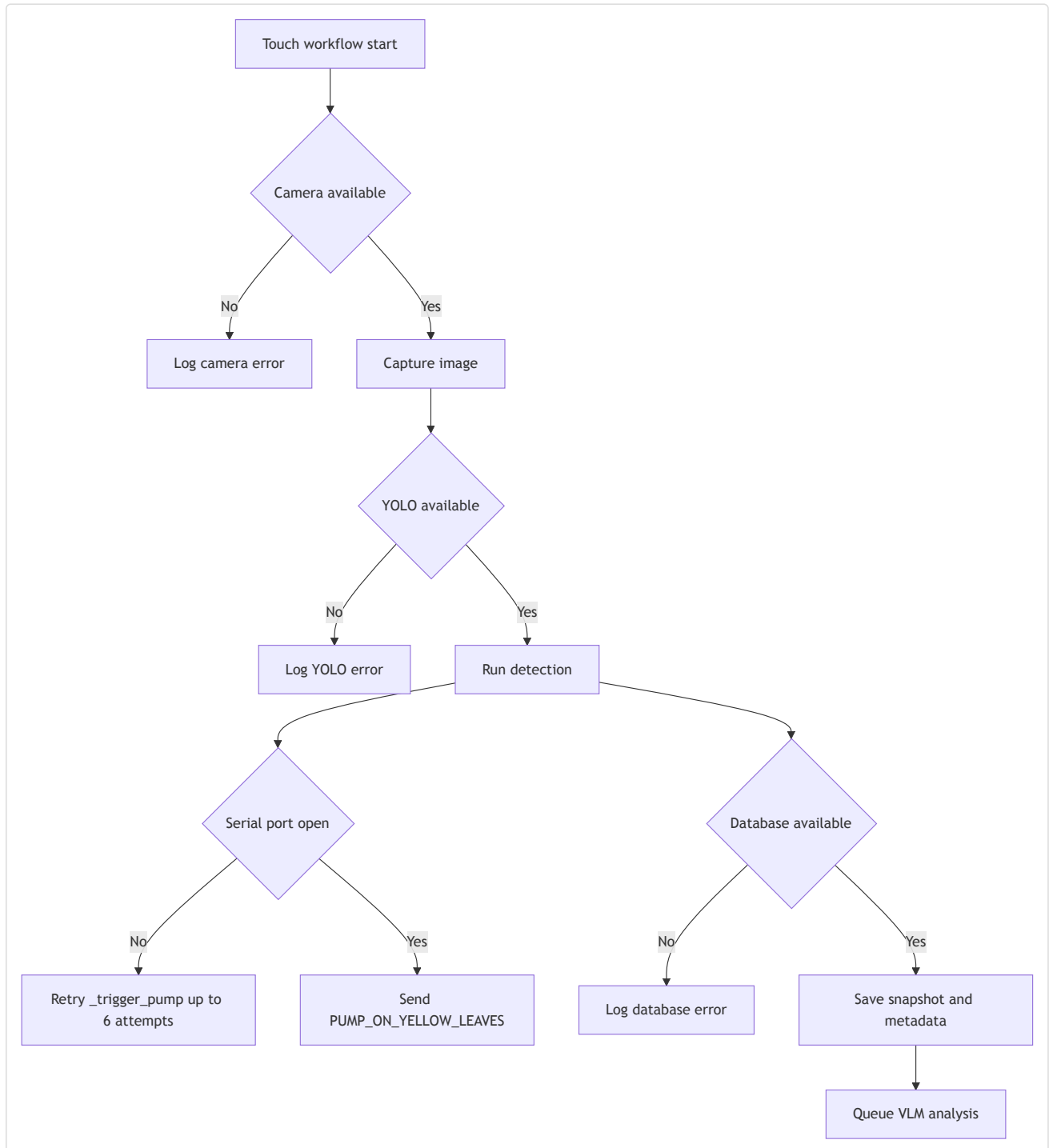


Figure 11: Error handling and resilience

- **Serial port:** The unified listener reconnects automatically if the port disappears, and the orchestrator retries pump commands up to six times when the port is transiently closed.
- **Database:** Each DB operation is wrapped in error handling with rollback. Sensor insert failures are logged but do not crash the listener.

- **Camera:** Capture failures are logged and skipped; the workflow continues to YOLO if an image is unavailable, then falls back to no detection.
- **YOLO:** If the model or Ultralytics is not available, the module returns `person_detected = false` so the workflow can still complete.
- **VLM:** If Ollama is offline, the worker marks jobs as failed with `analysis_error`. The system continues to capture and queue new snapshots for later analysis.
- **SD card:** Scripts fall back to local directories when the SD card path is not present or not writable.

16. API Reference

The FastAPI application in `app.py` plus the `touch_routes.py` and `routes/logs.py` routers expose the following endpoints. All times are returned as ISO 8601 strings. CORS is enabled for all origins to support the Vite dev server.

Method	Path	Description
GET	/health	Simple liveness check
POST	/api/readings	Insert a sensor reading manually
GET	/api/readings/latest	Latest sensor reading
GET	/api/readings/recent?limit=N	Recent sensor readings
GET	/api/readings/range	Readings in a time range
GET	/api/snapshots/latest	Most recent snapshot
GET	/api/snapshots/recent	List recent snapshots with image, boxed image, and video URLs
GET	/api/snapshots/latest-detection	Latest snapshot with person detection metadata
POST	/api/touch-event	Record a touch event
GET	/api/touch-event/latest	Latest touch event
GET	/api/touch-event/recent	Recent touch events
GET	/api/touch-event/status	Current touch status
GET	/api/analysis/latest-image	Latest snapshot with full VLM analysis
GET	/api/analysis/latest-video	Latest video event analysis
GET	/api/analysis/recent	Recent VLM analyses
GET	/api/analysis/health-alerts	Snapshots with plant health concerns
GET	/api/analysis/status	Ollama, model, and queue status
POST	/api/analysis/run-latest	Placeholder to trigger VLM on latest snapshot
GET	/api/logs/latest	Latest log lines
GET	/api/logs/files	List log files
GET	/api/logs/file/{filename}	Read a specific log file
GET	/api/logs/stats	Aggregate log stats
GET	/images/{path}	Static image files

Method	Path	Description
GET	/videos/{filename}	Video files with correct MIME type

17. Environment Variables

Behavior is configured through environment variables or a `.env` file in the `backend` directory. Defaults are used when a variable is not set.

Variable	Default	Purpose
DATABASE_URL	postgresql://plantuser:plantpass@localhost:5433/plantdb	Connection string to TimescaleDB
IMAGES_DIR	backend/images or SD card /images	Where snapshots are saved
VIDEOS_DIR	backend/videos or SD card /videos	Where video clips are saved
LOGS_DIR	backend/logs or SD card /logs	Where log files are written
DEFAULT_DEVICE_ID	ESP32_PLANT_01	Fallback device ID for sensor/touch events
CAMERA_AUTO_EXPOSURE	1	OpenCV auto exposure mode
CAMERA_EXPOSURE	-6	Exposure level
CAMERA_BRIGHTNESS	128	Brightness 0-255
CAMERA_CONTRAST	32	Contrast 0-100
CAMERA_WARMUP_TIME	0.5	Seconds to wait after setting camera
IMAGE_JPEG_QUALITY	85	JPEG quality for snapshots
SNAPSHOT_COOLDOWN	60	Minimum seconds between temperature snapshots
VIDEO_ALERT_ENABLED	true	Enable video on temperature alert
VIDEO_ALERT_DURATION	5	Seconds to record

Variable	Default	Purpose
VIDEO_ALERT_FPS	25	Frames per second for videos
TOUCH_VIDEO_ENABLED	true	Record a video on every touch event
OLLAMA_HOST	http://localhost:11434	Ollama server URL
OLLAMA_MODEL	llava:7b	Vision model name
OLLAMA_TIMEOUT	600	Seconds to wait for Ollama
VLM_ENABLED	true	Enable VLM processing
VLM_COOLDOWN_SECONDS	600	Cooldown between VLM runs in worker
VLM_POLL_INTERVAL	15	Worker polling interval
VLM_MAX_IMAGE_WIDTH	640	Resize image width for VLM
VLM_IMAGE_QUALITY	75	JPEG quality for VLM upload

18. Cross-Check Coverage Matrix

This matrix confirms that every major component is covered by at least one flow and that every flow is documented. An **X** means the component participates in the flow.

Component	Sensor Data Ingest	Touch Event	Pump Control	VLM Analysis	UI Query	Logging	Startup
ESP32 / firmware	X	X	X				
Capacitive touch sensor		X					
HDC302x	X						
Webcam		X					
Peristaltic pump			X				
serial_unified_listener.py	X	X				X	X
TouchWorkflowOrchestrator		X	X	X		X	
capture_snapshot.py		X				X	
capture_video.py		X				X	
detect_person.py		X	X			X	
YOLOv8n		X	X				
vlm_worker.py				X		X	X
vlm_analyzer.py				X			
analysis_rules.py				X			
ollama_client.py				X			
Ollama / llava:7b				X			
TimescaleDB	X	X		X	X		X
FastAPI / app.py		X			X		X
touch_routes.py		X			X		
routes/logs.py						X	
React frontend					X		
SD card storage		X				X	
tts_service.py		X					

Component	Sensor Data Ingest	Touch Event	Pump Control	VLM Analysis	UI Query	Logging	Startup
logging_config.py						X	

Every component listed in the Component Inventory appears in at least one flow, and every system flow (sensor ingestion, touch event, pump control, VLM analysis, UI query, logging, startup) is represented by a diagram and a textual description.

19. Key File Inventory

File	Purpose
plant-monitor/esp32/touch_sensor/touch_sensor.ino	Production ESP32 firmware
plant-monitor/backend/app.py	FastAPI application and REST endpoints
plant-monitor/backend/serial_unified_listener.py	Main serial reader and workflow trigger
plant-monitor/backend/serial_reader.py	Original sensor-only serial reader
plant-monitor/backend/serial_touch_listener.py	Dedicated touch-only serial listener
plant-monitor/backend/services/touch_workflow.py	Touch pipeline orchestrator
plant-monitor/backend/services/tts_service.py	macOS text-to-speech wrapper
plant-monitor/backend/capture_snapshot.py	Webcam image capture
plant-monitor/backend/capture_video.py	Webcam video recording and MP4 conversion
plant-monitor/backend/capture_with_vlm.py	End-to-end capture, YOLO, and DB insert
plant-monitor/backend/detect_person.py	YOLOv8n person detection
plant-monitor/backend/vlm/vlm_analyzer.py	VLM image and video analysis
plant-monitor/backend/vlm/ollama_client.py	Ollama REST client
plant-monitor/backend/vlm/analysis_rules.py	VLM output normalization and rules
plant-monitor/backend/vlm/prompt_templates.py	VLM prompts
plant-monitor/backend/vlm/video_frame_sampler.py	Video frame extraction for VLM
plant-monitor/backend/vlm_worker.py	Background VLM worker
plant-monitor/backend/touch_routes.py	Touch API routes
plant-monitor/backend/routes/logs.py	Logs API routes
plant-monitor/backend/logging_config.py	Centralized logging setup
plant-monitor/backend/schema.sql	Initial database schema
plant-monitor/backend/migrations/00*.sql	Schema migrations for detection, video, VLM, and touch events
plant-monitor/backend/start_all.sh	Starts database, backend, serial listener, VLM worker, and frontend
plant-monitor/backend/start_backend.sh	Starts FastAPI with venv and SD path detection

File	Purpose
plant-monitor/backend/start_unified_listener.sh	Starts serial_unified_listener.py
plant-monitor/backend/start_vlm_worker.sh	Starts vlm_worker.py
plant-monitor/backend/docker-compose.yml	TimescaleDB container definition
plant-monitor/frontend/src/App.jsx	React dashboard entry component
plant-monitor/frontend/src/PlantHealthCard.jsx	Plant health display
plant-monitor/frontend/src/PlantSnapshot.jsx	Latest snapshot display
plant-monitor/frontend/src/SnapshotGrid.jsx	Snapshot gallery
plant-monitor/frontend/src/PersonDetection.jsx	Person detection display
plant-monitor/frontend/src/AIAnalysisCard.jsx	Image VLM analysis
plant-monitor/frontend/src/VideoAnalysisCard.jsx	Video VLM analysis
plant-monitor/frontend/src/AIStatusCard.jsx	Ollama and queue status
plant-monitor/frontend/src/TouchStatusCard.jsx	Touch state card
plant-monitor/frontend/src/components/LogsPanel.jsx	Live log viewer

Additional documentation, prototype sketches, and test programs are present in the repository root, including README.md, VLM_INTEGRATION_GUIDE.md, TOUCH_SENSOR_INTEGRATION_COMPLETE.md, and the firmware test folders.